



L OVELY
P ROFESSIONAL
U NIVERSITY

Transforming Education Transforming India

Minor Project Report

on

E-Commerce Database Management

Course: Advanced Database Techniques

Course Code: CAP570

Submitted by

1. Abhay Pratap singh (12415511), Group: - 1
2. Maibam Ajay Meitei (12403416), Group: -1
3. Om Shree Jaiswal (12414369), Group: -1

Submitted to

Ms. Ranjit Kaur Walia
UID: 28632
Assistant Professor, SCA, LPU

Lovely Faculty of Technology & Sciences

School of Computer Applications

Lovely Professional University

Punjab

Acknowledgment

We would like to express our sincere gratitude to **Ms. Ranjit Kaur Walia (28632), Assistant Professor, School of Computer Applications, Lovely Professional University**, for her guidance and continuous supervision, as well as for providing essential information and unwavering support throughout the project. Her consistent direction and willingness to share her extensive knowledge enabled us to gain a deep understanding of the project, which greatly assisted us in completing our tasks on time.

We would also like to extend our heartfelt thanks to all the individuals who graciously participated in the interviews and surveys conducted during the project. Their experiences and insights provided a practical understanding of how database systems are used in real-world applications, which greatly enriched the quality of this project.

Finally, we would like to thank our families and friends for their constant encouragement and support during this journey. This project would not have been possible without the contribution of everyone involved.

Thank you all.

Sr. No.	Content	Page No.
1.	Introduction	4
2.	Project Overview	4
3.	Project Objectives	5
4.	Problem Statement	5
5.	Requirement Gathering	6 - 7
	5.1 Interviews and Surveys	6
	5.2 Website Exploration	7
6.	ER Diagram and Relational Schema	8 - 10
7.	Database Design and Normalization	10 - 11
	7.1 Tables Description	10 - 11
	7.2 Normalization Process	11
8.	SQL Queries and Database Operations	12 - 31
	8.1 Creating Tables	12 - 15
	8.2 Inserting Data	16 – 18
	8.3 Queries for Data Retrieval	19 - 31
9.	Conclusion	31

1. Introduction

This project aims to create a robust database management system for an e-commerce platform. The system will handle multiple core aspects such as customer management, product information, orders, and shipping details. It ensures efficient handling of customer and product data, essential for maintaining seamless operations in an online marketplace.

E-commerce platforms require efficient database systems to handle large volumes of data, ranging from user information to order details, similar to platforms like Amazon or Flipkart.

2. Project Overview

This project revolves around building a database to manage an e-commerce site's core functions. The database will manage customers, products, sellers, orders, and shipping information.

The database structure includes several entities:

- **Address:** Stores customer and seller addresses.
- **Customers:** Information about customers.
- **Orders:** Records of customer orders.
- **Payment:** Details of the payment of the orders.
- **Products:** Product details such as price, category, and seller information.
- **Ordered Product Details:** Information about ordered products.
- **Sellers:** Data about sellers, including ratings and contact information.
- **Product Categories:** The categorization of products.
- **Category Type:** The parent category of the product category.
- **Shipped Products:** Information about shipping status, tracking, and delivery.
- **Returned and Exchanged Products:** Managing requests for returns or exchanges.
- **Cart:** Stores the products that are selected by the customers.

3. Project Objectives

The objectives of this project include:

- Designing a scalable and normalized database structure for e-commerce.
- Efficiently managing customer, product, and seller data.
- Streamlining order processing and tracking shipment and delivery.
- Ensuring easy retrieval of order and customer information for future analysis.
- Handling returned and exchanged products with proper documentation.

4. Problem Statement

The core problem being addressed is the need for efficient management of customer, product, and order data in an e-commerce system. As online marketplaces grow, managing large-scale product and customer databases becomes challenging. This system solves the following:

- Easy retrieval of customer orders and history.
- Efficient tracking of products from order placement to delivery.
- Handling returns and exchanges seamlessly.

5. Requirement Gathering

5.1 Interviews and Surveys

To gather requirements for this project, interviews were conducted with professionals from e-commerce platforms and database administrators. The feedback included common challenges in scaling databases for e-commerce, managing customer returns, and handling large product inventories.



5.2 Website Exploration

Various e-commerce platforms like Amazon and Flipkart were explored to understand how similar systems manage product and customer data. Observations were made on user profiles, product categories, and the order and return flow. Screenshots were captured from these platforms to study their features.

The screenshot shows the Amazon India Shopping Cart interface. At the top, the Amazon.in logo is on the left, and the delivery location is set to Phagwara 144411. A search bar is in the center, and navigation links for account, orders, and cart are on the right. Below the header, a horizontal menu lists various categories like Fresh, MX Player, Sell, etc. The main content area is titled 'Shopping Cart' and lists two items: 'boAt Airdopes 141 Bluetooth Truly Wireless in Ear Ear Buds' for ₹1,299.00 and 'Arctic Fox Pureview Transparent Wireless and Bluetooth Rechargeable Mouse' for ₹627.00. Each item has a product image, description, stock status, and shipping eligibility. To the right of the items, a summary shows a subtotal of ₹1,926.00 and a 'Proceed to Buy' button. Below the summary, a section titled 'Customers who bought items in your Recent History also bought' features a recommended product, the 'Source Earphone Case', for ₹209.00.

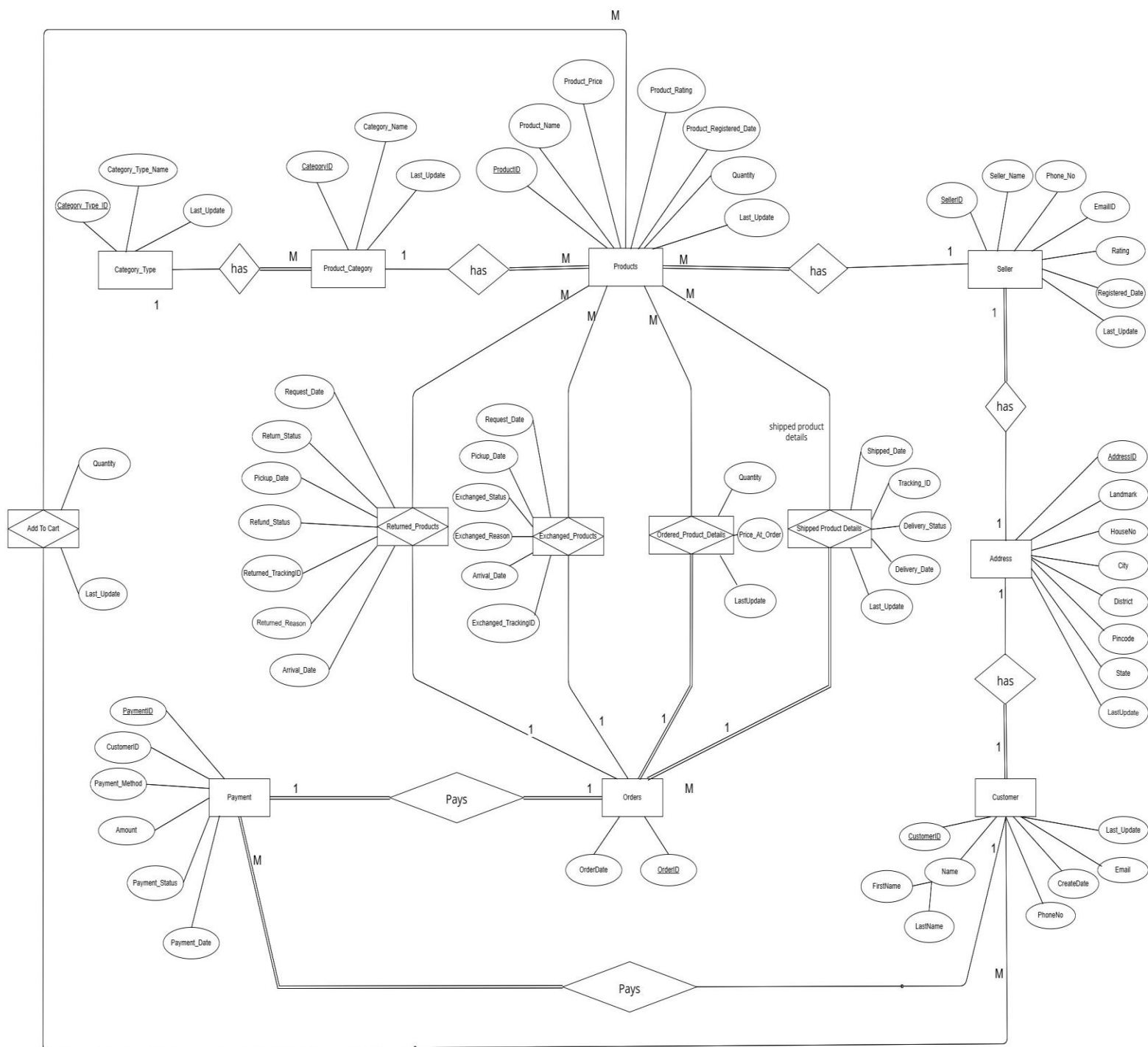
The screenshot displays the Flipkart homepage. The top navigation bar includes the Flipkart logo, a search bar, and links for login, cart, and becoming a seller. Below the navigation bar, a horizontal menu categorizes products into Grocery, Mobiles, Fashion, Electronics, Home & Furniture, Appliances, Flight Bookings, Beauty, Toys & More, and Two Wheelers. A large promotional banner for 'Flipkart Flights' features an image of an Air India plane and the text 'Escape. Explore. Experience! Air India Flights From ₹1,499*'. Below the banner, a section titled 'Best of Electronics' showcases a grid of product images, including earbuds, a smartwatch, a laptop, a camera, and a drone.

6. ER Diagram and Relational Schema

ER Diagram

The ER Diagram captures the relationship between different entities:

- **Customers** can place multiple **Orders**.
- Each **Order** consists of multiple **Products**.
- **Sellers** provide **Products** to the platform.
- **Shipped Products** are tracked after orders are placed.
- **Returned and Exchanged Products** are managed through the system.



Relational Schema

- **Address**(AddressID, Landmark, HouseNO, District, City, Postalcode, Last_update)
- **Customer**(CustomersID, FirstName, LastName, PhoneNumber, Email, AddressID, CreateDate, Last_update)
- **Orders**(OrderID, OrderDate, PaymentID)
- **Category_Type**(CategoryTypeID, CategoryTypeName, Last_Update)
- **Product_Category**(CategoryID, CategoryName, CategoryTypeID, Last_Update)
- **Payment**(PaymentID, CustomerID, Payment_Method, Amount, Payment_Status, Payment_Date)
- **Products**(ProductID, ProductName, ProductPrice, CategoryID, ProductRating, SellerID, Quantity, ProductRegisteredDate, Last_update)
- **Cart**(CustomerID, ProductID, Quantity, Last_Update)
- **Ordered_Product_Details**(OrderID, ProductID, Price_At_Order, Quantity, Last_update)
- **Sellers**(SellerID, SellerName, PhoneNumber, Email, Rating, AddressID, RegisteredDate, LastUpdate)
- **Shipped_Products_Details**(OrderID, ProductID, ShippedDate, TrackingID, DeliveryStatus, DeliveryDate, LastUpdate)
- **Returned_Products**(OrderID, ProductID, RequestDate, Returned_Status, PickedUpDate, Refund_Status, ReturnedTrackingID, ReturnedReason, ArrivalDate)
- **Exchanged_Products**(OrderID, ProductID, RequestDate, Exchanged_Status, PickedUpDate, ExchangedTrackingID, ExchangedReason, ArrivalDate)

7. Database Design and Normalization

7.1 Table Descriptions

- **Address:** Stores detailed addresses for both customers and sellers, including landmarks, house numbers, districts, cities, states, and pincodes.
- **Customers:** Contains information about customers, including personal details like names, phone numbers, emails, and associated addresses.
- **Orders:** Records all customer orders, including order IDs, order dates, and statuses, linking customers to their respective purchases.
- **Payment:** Details the payment transactions for orders, including payment methods, customer IDs, amounts, statuses, and dates, ensuring all financial activities are documented.
- **Products:** Holds information about products available for sale, including pricing, quantity, descriptions, categories, and seller details.
- **Ordered Product Details:** Provides detailed information about products within each order, linking products to specific orders and capturing quantities and statuses.
- **Sellers:** Contains data about sellers, including their names, contact information, ratings, and other relevant attributes, facilitating a connection between products and their sellers.
- **Product Categories:** Classifies products into various categories for easier navigation and organization, linking each product to its respective category.

- **Category Type:** Represents the hierarchical structure of product categories, indicating parent-child relationships between different categories.
- **Shipped Products:** Tracks the shipping status of products, including tracking IDs, shipment dates, and delivery statuses, to manage the logistics of product delivery.
- **Returned and Exchanged Products:** Manages requests for returning or exchanging products, capturing reasons for returns, statuses, and related details to facilitate customer service.
- **Cart:** Stores information about products selected by customers for potential purchase, allowing for temporary holding of items before finalizing an order.

7.2 Normalization Process

- **1NF:** Ensured atomicity by separating multi-valued attributes into individual tables.
- **2NF:** Removed partial dependencies by splitting composite keys.
- **3NF:** Eliminated transitive dependencies, ensuring all attributes depend only on the primary key.
- ❖ The details of the payments and ordered products cannot be in the Orders table. So, separate tables Payment and Order Product Details are created.
- ❖ Addresses of customers and sellers cannot be in their respective tables. So, a separate table of Address is created.
- ❖ A separate Category Type table is created to have the details of parent category type of the product type.

8. SQL Queries and Database Operations

8.1 Creating Tables

8.1.1 Address

SQL Server Code:

```
CREATE TABLE Address (  
  AddressID INT IDENTITY(1,1),  
  Landmark VARCHAR(100) NOT NULL,  
  HouseNumber VARCHAR(20) NOT NULL,  
  District VARCHAR(50) NOT NULL,  
  City VARCHAR(50) NOT NULL,  
  State VARCHAR(50) NOT NULL,  
  Pincode VARCHAR(20) NOT NULL,  
  Last_Update DATETIME NOT NULL,  
  CONSTRAINT AddrssID_PK PRIMARY KEY(AddressID) );
```

8.1.2 Customer

SQL Server Code:

```
CREATE TABLE Customer (  
  CustomerID INT IDENTITY(100,1),  
  FirstName VARCHAR(50) NOT NULL,  
  LastName VARCHAR(50) NOT NULL,  
  PhoneNumber VARCHAR(15) NOT NULL,  
  Email VARCHAR(100) NOT NULL,  
  AddressID INT FOREIGN KEY REFERENCES ADDRESS(AddressID) NOT NULL,  
  CreateDate DATETIME  
  Last_Update DATETIME NOT NULL,  
  CONSTRAINT CustmrID_PK PRIMARY KEY(CustomerID) );
```

8.1.3 Orders

SQL Server Code:

```
CREATE TABLE Orders (  
  OrderID INT IDENTITY(1,1),  
  OrderDate DATETIME DEFAULT(GETDATE()),  
  PaymentID INT NOT NULL,  
  CONSTRAINT OrderID_PK PRIMARY KEY(OrderID),  
  CONSTRAINT PaymentID_FK FOREIGN KEY(PaymentID) REFERENCES  
  Payment(PaymentID) );
```

8.1.4 Category_Type

SQL Server Code:

```
CREATE TABLE Category_Type (  
    CategoryTypeID INT IDENTITY(1,1),  
    CategoryTypeName VARCHAR(50) NOT NULL,  
    Last_Update DATETIME NOT NULL,  
    CONSTRAINT CategoryTypeID_PK PRIMARY KEY (CategoryTypeID) );
```

8.1.5 Product_Category

SQL Server Code:

```
CREATE TABLE Product_Category (  
    CategoryID INT IDENTITY(1,1),  
    CategoryName VARCHAR(50) NOT NULL,  
    CategoryTypeID INT,  
    Last_update DATETIME NOT NULL,  
    CONSTRAINT CategoryID_PK PRIMARY KEY(CategoryID),  
    CONSTRAINT CategoryTypeID_FK FOREIGN KEY(CategoryTypeID) REFERENCES  
    Category_Type(CategoryTypeID) );
```

8.1.6 Payment

SQL Server Code:

```
CREATE TABLE Payment (  
    PaymentID INT IDENTITY(1,1),  
    CustomerID INT NOT NULL,  
    Payment_Method VARCHAR(30) NOT NULL,  
    Amount FLOAT NOT NULL,  
    Payment_Status VARCHAR(30),  
    Payment_Date DATETIME DEFAULT(GETDATE()),  
    CONSTRAINT PaymntID_PK PRIMARY KEY(PaymentID),  
    CONSTRAINT PayCustID_FK FOREIGN KEY(CustomerID) REFERENCES  
    Customer(CustomerID) );
```

8.1.7 Seller

SQL Server Code:

```
CREATE TABLE Seller (  
    SellerID INT IDENTITY(1,1),  
    Seller_Name VARCHAR(100),  
    Phone_Number VARCHAR(15),  
    EmailID VARCHAR(100),  
    Rating DECIMAL(2, 1),  
    AddressID INT,  
    Registered_Date DATETIME DEFAULT(GETDATE()),  
    Last_Update DATETIME,  
    CONSTRAINT SellerID_PK PRIMARY KEY(SellerID),  
    CONSTRAINT AddressID_FK FOREIGN KEY(AddressID) REFERENCES  
    Address(AddressID) );
```

8.1.8 Products

SQL Server Code:

```
CREATE TABLE Products (
ProductID INT IDENTITY(1,1),
Product_Name VARCHAR(100) NOT NULL,
Product_Price DECIMAL(10, 2) NOT NULL,
CategoryID INT,
Product_Rating DECIMAL(2, 1),
Quantity INT,
SellerID INT,
Product_Registered_Date DATETIME DEFAULT(GETDATE()),
Last_Update DATETIME NOT NULL,
CONSTRAINT ProductID_PK PRIMARY KEY(ProductID),
CONSTRAINT CategoryID_FK FOREIGN KEY(CategoryID) REFERENCES
PRODUCT_CATEGORY(CategoryID),
CONSTRAINT SellerID_FK FOREIGN KEY(SellerID) REFERENCES Seller(SellerID) );
```

8.1.9 Ordered_Product_Details

SQL Server Code:

```
CREATE TABLE Ordered_Product_Details (
OrderID INT,
ProductID INT,
Price_At_Order FLOAT,
Quantity INT NOT NULL,
Last_update DATETIME DEFAULT(GETDATE()),
CONSTRAINT OrderID_ProductID_PK PRIMARY KEY (OrderID, ProductID),
CONSTRAINT OrderID_FK FOREIGN KEY(OrderID) REFERENCES ORDERS(OrderID),
CONSTRAINT ProductID_FK FOREIGN KEY(ProductID) REFERENCES
Products(ProductID) );
```

8.1.10 Shipped_Products_Details

SQL Server Code:

```
CREATE TABLE Shipped_Products_Details (
OrderID INT,
ProductID INT,
Shipped_Date DATETIME DEFAULT(GETDATE()),
Tracking_ID INT IDENTITY(1000,1),
Delivery_Status VARCHAR(50) NOT NULL,
Delivery_Date DATETIME,
Last_Update DATETIME NOT NULL,
CONSTRAINT SHPD_OrID_ProID_PK PRIMARY KEY (OrderID, ProductID),
CONSTRAINT SHPD_OrID_FK FOREIGN KEY(OrderID) REFERENCES ORDERS(OrderID),
CONSTRAINT SHPD_ProID_FK FOREIGN KEY(ProductID) REFERENCES
Products(ProductID) );
```

8.1.11 Returned_Products

SQL Server Code:

```
CREATE TABLE Returned_Products (
  OrderID INT,
  ProductID INT,
  Request_Date DATETIME DEFAULT(GETDATE()),
  Returned_Status VARCHAR(50) NOT NULL,
  Pickup_Date DATETIME NOT NULL,
  Refund_Status VARCHAR(50) NOT NULL,
  Returned_TrackingID VARCHAR(50) NOT NULL,
  Returned_Reason VARCHAR(255) NOT NULL,
  Arrival_Date DATETIME,
  CONSTRAINT RETRN_OrID_ProID_PK PRIMARY KEY (OrderID, ProductID),
  CONSTRAINT RETRN_OrID_FK FOREIGN KEY(OrderID) REFERENCES ORDERS(OrderID),
  CONSTRAINT RETRN_ProID_FK FOREIGN KEY(ProductID) REFERENCES
  Products(ProductID) );
```

8.1.12 Exchanged_products

SQL Server Code:

```
CREATE TABLE Exchanged_products (
  OrderID INT,
  ProductID INT,
  Request_Date DATETIME DEFAULT(GETDATE()),
  Exchanged_Status VARCHAR(50) NOT NULL,
  Pickup_Date DATETIME,
  Exchanged_TrackingID VARCHAR(50),
  Exchanged_Reason VARCHAR(255) NOT NULL,
  Arrival_Date DATETIME,
  CONSTRAINT XCHNG_OrID_ProID_PK PRIMARY KEY (OrderID, ProductID),
  CONSTRAINT XCHNG_OrID_FK FOREIGN KEY(OrderID) REFERENCES ORDERS(OrderID),
  CONSTRAINT XCHNG_ProID_FK FOREIGN KEY(ProductID) REFERENCES
  Products(ProductID) );
```

8.1.13 Cart

SQL Server Code:

```
CREATE TABLE Cart (
  CustomerID INT,
  ProductID INT,
  Quantity INT NOT NULL,
  Last_Update DATETIME NOT NULL,
  CONSTRAINT CART_CUST_PRO_ID_PK PRIMARY KEY (CustomerID, ProductID),
  CONSTRAINT CART_CUST_ID_FK FOREIGN KEY(CustomerID) REFERENCES
  Customer(CustomerID),
  CONSTRAINT CART_PROD_ID_FK FOREIGN KEY(ProductID) REFERENCES
  Products(ProductID) );
```


8.2 Inserting Data

8.2.1 Address

```
INSERT INTO Address(Landmark, HouseNumber, District, City, State, Pincode, Last_Update)
VALUES
('Near Park', '123', 'Mumbai', 'Mumbai', 'Maharashtra', '400001', GETDATE()),
('Opposite Mall', '456', 'Pune', 'Pune', 'Maharashtra', '411001', GETDATE()),
('Beside School', '789', 'Bengaluru', 'Bengaluru', 'Karnataka', '560001', GETDATE()),
('Near Hospital', '321', 'Chennai', 'Chennai', 'Tamil Nadu', '600001', GETDATE()),
('In Front of Temple', '654', 'Hyderabad', 'Hyderabad', 'Telangana', '500001', GETDATE()),
('Near Railway Station', '987', 'Delhi', 'New Delhi', 'Delhi', '110001', GETDATE()),
('Beside Market', '147', 'Ahmedabad', 'Ahmedabad', 'Gujarat', '380001', GETDATE()),
('Near Bus Stand', '258', 'Kolkata', 'Kolkata', 'West Bengal', '700001', GETDATE()),
('Next to College', '369', 'Jaipur', 'Jaipur', 'Rajasthan', '302001', GETDATE()),
('By the River', '852', 'Lucknow', 'Lucknow', 'Uttar Pradesh', '226001', GETDATE()),
('Across the Street', '963', 'Coimbatore', 'Coimbatore', 'Tamil Nadu', '641001',
GETDATE()),
('Near City Center', '741', 'Bhopal', 'Bhopal', 'Madhya Pradesh', '462001', GETDATE()),
('Adjacent to Garden', '258', 'Chandigarh', 'Chandigarh', 'Chandigarh', '160001',
GETDATE()),
('By the Beach', '159', 'Visakhapatnam', 'Visakhapatnam', 'Andhra Pradesh', '530001',
GETDATE()),
('In the Old City', '753', 'Varanasi', 'Varanasi', 'Uttar Pradesh', '221001', GETDATE()),
('Close to Stadium', '159', 'Surat', 'Surat', 'Gujarat', '395001', GETDATE()),
('At the Crossroads', '357', 'Patna', 'Patna', 'Bihar', '800001', GETDATE()),
('Near Airport', '951', 'Nashik', 'Nashik', 'Maharashtra', '422001', GETDATE()),
('In the Town Square', '753', 'Indore', 'Indore', 'Madhya Pradesh', '452001', GETDATE()),
('Close to the Fort', '258', 'Agra', 'Agra', 'Uttar Pradesh', '282001', GETDATE());
```

8.2.2 Customer

```
INSERT INTO Customer (FirstName, LastName, PhoneNumber, Email, AddressID, Last_Update)
VALUES
('Vivaan', 'Gupta', '8765432109', 'vivaan.gupta@example.com', 2, GETDATE()),
('Aditya', 'Singh', '7654321098', 'aditya.singh@example.com', 3, GETDATE()),
('Vihaan', 'Reddy', '6543210987', 'vihaan.reddy@example.com', 4, GETDATE()),
('Arjun', 'Kumar', '5432109876', 'arjun.kumar@example.com', 5, GETDATE()),
('Sai', 'Patel', '4321098765', 'sai.patel@example.com', 6, GETDATE()),
('Reyansh', 'Malhotra', '3210987654', 'reyansh.malhotra@example.com', 7, GETDATE()),
('Krishna', 'Nair', '2109876543', 'krishna.nair@example.com', 8, GETDATE()),
('Kabir', 'Verma', '1098765432', 'kabir.verma@example.com', 9, GETDATE()),
('Shivansh', 'Chopra', '0987654321', 'shivansh.chopra@example.com', 10, GETDATE()),
('Rohan', 'Joshi', '9871234567', 'rohan.joshi@example.com', 11, GETDATE()),
('Arnav', 'Bansal', '8762345678', 'arnav.bansal@example.com', 12, GETDATE()),
('Ayaan', 'Sethi', '7653456789', 'ayaan.sethi@example.com', 13, GETDATE()),
('Ansh', 'Yadav', '6544567890', 'ansh.yadav@example.com', 14, GETDATE()),
('Kian', 'Dixit', '5435678901', 'kian.dixit@example.com', 15, GETDATE()),
('Nirvan', 'Sinha', '4326789012', 'nirvan.sinha@example.com', 16, GETDATE()),
('Darsh', 'Khan', '3217890123', 'darsh.khan@example.com', 17, GETDATE()),
('Lakshya', 'Mishra', '2108901234', 'lakshya.mishra@example.com', 18, GETDATE()),
('Rudra', 'Puri', '1099012345', 'rudra.puri@example.com', 19, GETDATE()),
('Yash', 'Saxena', '0980123456', 'yash.saxena@example.com', 20, GETDATE());
```

8.2.3 Orders

```
INSERT INTO Orders (PaymentID)
VALUES
(1),
(2),
(3);
```

8.2.4 Category_Type

```
INSERT INTO CATEGORY_TYPE
VALUES
('Gadgets', GETDATE()),
('Home Appliances', GETDATE()),
('Accessories', GETDATE()),
('Wearables', GETDATE()),
('Tablets', GETDATE()),
('Cameras', GETDATE()),
('Printers', GETDATE()),
('Monitors', GETDATE()),
('Keyboards', GETDATE()),
('Mice', GETDATE()),
('Televisions', GETDATE()),
('Speakers', GETDATE()),
('Networking Devices', GETDATE()),
('Smart Home', GETDATE()),
('Chargers', GETDATE()),
('Drones', GETDATE()),
('Virtual Reality', GETDATE()),
('Gaming Consoles', GETDATE()),
('Fitness Devices', GETDATE()),
('E-readers', GETDATE())
```

8.2.5 Product_Category

```
INSERT INTO PRODUCT_CATEGORY (CategoryName, CategoryTypeID, Last_update)
VALUES
('Electronics', 1, GETDATE()),
('Home Appliances', 2, GETDATE()),
('Accessories', 3, GETDATE()),
('Wearables', 4, GETDATE()),
('Tablets', 5, GETDATE()),
('Cameras', 6, GETDATE()),
('Printers', 7, GETDATE()),
('Monitors', 8, GETDATE()),
('Keyboards', 9, GETDATE()),
('Mice', 10, GETDATE()),
('Televisions', 11, GETDATE()),
('Speakers', 12, GETDATE()),
('Networking Devices', 13, GETDATE()),
('Smart Home', 14, GETDATE()),
('Smart Plugs', 15, GETDATE()),
('Drones', 16, GETDATE()),
('Virtual Reality', 17, GETDATE()),
('Gaming Consoles', 18, GETDATE()),
('Chargers', 19, GETDATE()),
```

8.2.6 Payment

```
INSERT INTO Payment (CustomerID, Payment_Method, Amount, Payment_Status)
VALUES
(100, 'Credit Card', 1500.00, 'Completed'),
(104, 'Debit Card', 2000.00, 'Pending'),
(105, 'Net Banking', 750.50, 'Failed');
```

8.2.7 Seller

```
INSERT INTO SELLER (Seller_Name, Phone_Number, EmailID, Rating, AddressID, Last_Update)
VALUES
('TechStore', '555-1011', 'techstore@example.com', 4.5, 1, GETDATE()),
('SmartDeals', '555-1012', 'smartdeals@example.com', 4.4, 2, GETDATE()),
('BestBuy', '555-1013', 'bestbuy@example.com', 4.6, 3, GETDATE()),
('GadgetWorld', '555-1014', 'gadgetworld@example.com', 4.7, 4, GETDATE()),
('SuperTech', '555-1015', 'supertech@example.com', 4.3, 5, GETDATE()),
('ElectroHub', '555-1016', 'electrohub@example.com', 4.5, 6, GETDATE()),
('GizmoPlace', '555-1017', 'gizmoplace@example.com', 4.4, 7, GETDATE()),
('DeviceSpot', '555-1018', 'devicespot@example.com', 4.2, 8, GETDATE()),
('ModernTech', '555-1019', 'moderntech@example.com', 4.8, 9, GETDATE()),
('PrimeElectronics', '555-1020', 'primeelectronics@example.com', 4.7, 10, GETDATE()),
('TopGadgets', '555-1021', 'topgadgets@example.com', 4.9, 11, GETDATE()),
('GadgetSpot', '555-1022', 'gadgetsport@example.com', 4.6, 12, GETDATE()),
('TechHub', '555-1023', 'techhub@example.com', 4.7, 13, GETDATE()),
('SmartMart', '555-1024', 'smartmart@example.com', 4.3, 14, GETDATE()),
('ElectroMart', '555-1025', 'electromart@example.com', 4.4, 15, GETDATE()),
('GadgetMart', '555-1026', 'gadgetmart@example.com', 4.5, 16, GETDATE()),
('DeviceMart', '555-1027', 'devicemart@example.com', 4.6, 17, GETDATE()),
('ModernStore', '555-1028', 'modernstore@example.com', 4.7, 18, GETDATE()),
('PrimeStore', '555-1029', 'primestore@example.com', 4.2, 19, GETDATE()),
('SuperGadgets', '555-1030', 'supergadgets@example.com', 4.3, 20, GETDATE());
```

8.2.8 Products

```
INSERT INTO Products (Product_Name, Product_Price, CategoryID, Product_Rating, Quantity,
SellerID, Last_Update)
VALUES
('Laptop', 999.99, 1, 4.5, 46, 1, GETDATE()),
('Smartphone', 699.99, 2, 4.3, 60, 2, GETDATE()),
('Headphones', 49.99, 3, 4.2, 40, 3, GETDATE()),
('Smartwatch', 199.99, 4, 4.1, 70, 4, GETDATE()),
('Tablet', 399.99, 5, 4.4, 87, 5, GETDATE()),
('Camera', 599.99, 6, 4.6, 92, 6, GETDATE()),
('Printer', 129.99, 7, 4.0, 46, 7, GETDATE()),
('Monitor', 299.99, 8, 4.7, 68, 8, GETDATE()),
('Keyboard', 29.99, 9, 4.2, 150, 9, GETDATE()),
('Mouse', 19.99, 10, 4.3, 12, 10, GETDATE()),
('TV', 799.99, 11, 4.5, 45, 11, GETDATE()),
('Speaker', 149.99, 12, 4.6, 70, 12, GETDATE()),
('Router', 79.99, 13, 4.1, 92, 13, GETDATE()),
('Smart Bulb', 39.99, 14, 4.4, 92, 14, GETDATE()),
('Smart Plug', 24.99, 15, 4.2, 46, 15, GETDATE()),
('Drone', 499.99, 16, 4.7, 46, 16, GETDATE()),
('VR Headset', 349.99, 17, 4.8, 46, 17, GETDATE()),
('Gaming Console', 599.99, 18, 4.9, 46, 18, GETDATE());
```

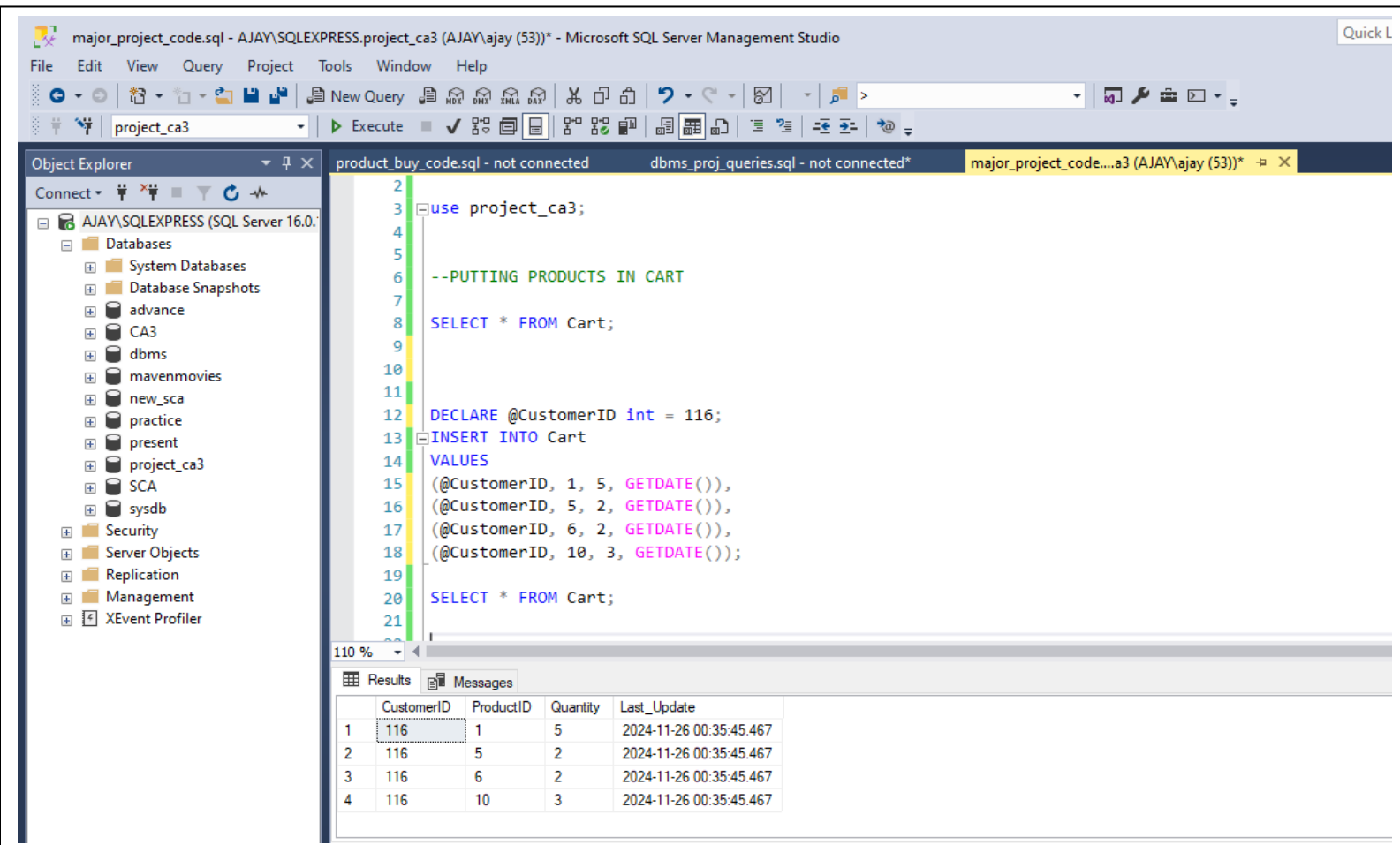
8.3. Queries for data retrieval

8.3.1 Putting products in cart

Code:

```
DECLARE @CustomerID int = 116;

--Inserting products in the Cart table
INSERT INTO Cart
VALUES
(@CustomerID, 1, 5, GETDATE()),
(@CustomerID, 5, 2, GETDATE()),
(@CustomerID, 6, 2, GETDATE()),
(@CustomerID, 10, 3, GETDATE());
```



The screenshot displays the Microsoft SQL Server Management Studio interface. The title bar indicates the connection to 'major_project_code.sql - AJAY\SQLEXPRESS,project_ca3 (AJAY\ajay (53))*'. The Object Explorer on the left shows the database structure, including 'Databases' and 'project_ca3'. The main query editor shows the following SQL script:

```
2
3 use project_ca3;
4
5
6 --PUTTING PRODUCTS IN CART
7
8 SELECT * FROM Cart;
9
10
11
12 DECLARE @CustomerID int = 116;
13 INSERT INTO Cart
14 VALUES
15 (@CustomerID, 1, 5, GETDATE()),
16 (@CustomerID, 5, 2, GETDATE()),
17 (@CustomerID, 6, 2, GETDATE()),
18 (@CustomerID, 10, 3, GETDATE());
19
20 SELECT * FROM Cart;
21
```

The 'Results' tab at the bottom shows the output of the query, displaying a table with four columns: CustomerID, ProductID, Quantity, and Last_Update. The data is as follows:

	CustomerID	ProductID	Quantity	Last_Update
1	116	1	5	2024-11-26 00:35:45.467
2	116	5	2	2024-11-26 00:35:45.467
3	116	6	2	2024-11-26 00:35:45.467
4	116	10	3	2024-11-26 00:35:45.467

8.3.2 Ordering products

Code:

```
--TRIGGER AFTER INSERTING ON PAYMENT TABLE
CREATE TRIGGER after_insert_payment ON Payment
AFTER INSERT
AS

BEGIN
INSERT INTO ORDERS(CustomerID, OrderDate, PaymentID)
SELECT CustomerID, GETDATE(), PaymentID from inserted;
END;

--TRIGGER AFTER INSERTING ON ORDER TABLE
CREATE TRIGGER after_insert_ORDER ON ORDERS
AFTER INSERT
AS

BEGIN
INSERT INTO Ordered_Product_Details(OrderID, ProductID, Price_At_Order, Quantity,
Last_update)
SELECT (SELECT OrderID FROM INSERTED), p.ProductID, p.Product_Price, c.Quantity,
GETDATE() FROM Cart c
Inner join Products p ON c.ProductId = p.ProductID
END;

--TRIGGER FOR DELETING THE ORDERED PRODUCT QUANTITY FROM THE PRODUCT TABLE
CREATE TRIGGER after_insert_Ordered_Product_Details ON Ordered_Product_Details
AFTER INSERT
AS

BEGIN
UPDATE P
SET P.Quantity = CASE
WHEN P.Quantity >= C.Quantity THEN P.Quantity - C.Quantity
ELSE 0 -- set to 0 if product is insufficient
END
FROM Products P
JOIN Ordered_Product_Details C ON P.ProductID = C.ProductID;
END;
```

```

-----
--Ordering products by the customers
-----
BEGIN TRANSACTION;

DECLARE @CustomerID int = 113;

--Inserting values to the Payment Table
INSERT INTO Payment(CustomerID, Payment_Method, Amount, Payment_Status,
Payment_Date)
Values(@CustomerID, 'upi', 400, 'paid', GETDATE());

--After insert payment triggers insert into Orders table
--After insert Orders triggers insert into Order_product_details table
--After insert Order_product_details triggers delete quantity from products table


--Emptying the cart of the customers
DELETE FROM Cart
WHERE CustomerID = @CustomerID;

COMMIT;
-----

```

major_project_code.sql - AJAY\SQLEXPRESS.project_ca3 (AJAY\ajay (53))* - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

project_ca3 Execute

Object Explorer

Connect

AJAY\SQLEXPRESS (SQL Server 16.0.)

- Databases
 - System Databases
 - Database Snapshots
 - advance
 - CA3
 - dbms
 - mavenmovies
 - new_sca
 - practice
 - present
 - project_ca3
 - SCA
 - sysdb
- Security
- Server Objects
- Replication
- Management
- XEvent Profiler

product_buy_code.sql - not connected dbms_proj_queries.sql - not connected* major_project_code....a3 (AJAY\ajay (53))*

```

91 --Inserting values to the Payment Table
92 INSERT INTO Payment(CustomerID, Payment_Method, Amount, Payment_Status, Payment_Date)
93 Values(@CustomerID,'upi', 800, 'paid', GETDATE());
94
95 --After insert payment triggers insert into Orders table
96 --After insert Orders triggers insert into Order_product_details table
97 --After insert Order_product_details triggers delete quantity from products table
98
99 --Emptying the cart of the customers
100 DELETE FROM Cart
101 WHERE CustomerID = @CustomerID;
102
103 COMMIT;
104
105
106 -----
107 -----
108
109 SELECT * FROM Payment;
110

```

110 %

Results Messages

	PaymentID	CustomerID	Payment_Method	Amount	Payment_Status	Payment_Date
7	7	104	upi	100	paid	2024-11-09 01:31:29.910
8	8	104	upi	100	paid	2024-11-09 01:35:51.010
9	9	110	upi	300	paid	2024-11-09 09:58:24.143
10	12	113	upi	400	paid	2024-11-25 22:24:39.743
11	13	116	upi	800	paid	2024-11-26 00:46:41.337

major_project_code.sql - AJAY\SQLEXPRESS,project_ca3 (AJAY\ajay (53))* - Microsoft SQL Server Management Studio

File Edit View Query Project Tools Window Help

project_ca3 Execute

Object Explorer

Connect

AJAY\SQLEXPRESS (SQL Server 16.0.)

- Databases
 - System Databases
 - Database Snapshots
 - advance
 - CA3
 - dbms
 - mavenmovies
 - new_sca
 - practice
 - present
 - project_ca3
 - SCA
 - sysdb
- Security
- Server Objects
- Replication
- Management
- XEvent Profiler

product_buy_code.sql - not connected dbms_proj_queries.sql - not connected* major_project_code.....a3 (AJAY\ajay (53))*

```
96 --After insert Orders triggers insert into Order_product_details table
97 --After insert Order_product_details triggers delete quantity from products table
98
99 --Emptying the cart of the customers
100 DELETE FROM Cart
101 WHERE CustomerID = @CustomerID;
102
103 COMMIT;
104
105
106 -----
107 -----
108
109 SELECT * FROM Payment;
110
111 SELECT * FROM Ordered_Product_Details;|
112
113
114 -----
115 --Shipping details from the seller
```

110 %

Results Messages

	OrderID	ProductID	Price_At_Order	Quantity	Last_update
26	13	20	99.99	4	2024-11-25 22:24:39.787
27	14	1	999.99	5	2024-11-26 00:46:41.387
28	14	5	399.99	2	2024-11-26 00:46:41.387
29	14	6	599.99	2	2024-11-26 00:46:41.387
30	14	10	19.99	3	2024-11-26 00:46:41.387

8.3.3 Shipping the products

Code:

```
--Stored Procedure to insert shipping details
CREATE PROCEDURE SHIPPING_DETAILS_INSERT
@Order_id INT
AS
BEGIN
DECLARE @Delivery_Status VARCHAR(30) = 'In transit';
INSERT INTO Shipped_Products_Details(OrderID, ProductID,
Delivery_Status,Last_Update)
SELECT OrderID, ProductID, @Delivery_Status, GETDATE() FROM
Ordered_Product_Details
WHERE OrderID = @Order_id;
END;

--Shipping details insertion transaction begins
BEGIN TRANSACTION;
DECLARE @ORDER_ID INT = 14;
EXEC SHIPPING_DETAILS_INSERT @Order_id = @ORDER_ID;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The left pane shows the Object Explorer with the 'project_ca3' database selected. The central pane shows the SQL Editor with the following code:

```
--Procedure to insert shipping details
119 CREATE PROCEDURE SHIPPING_DETAILS_INSERT
120 @Order_id INT
121 AS
122 BEGIN
123 DECLARE @Delivery_Status VARCHAR(30) = 'In transit';
124 INSERT INTO Shipped_Products_Details(OrderID, ProductID, Delivery_Status,Last_Update)
125 SELECT OrderID, ProductID, @Delivery_Status, GETDATE() FROM Ordered_Product_Details
126 WHERE OrderID = @Order_id;
127 END;
128
129
130
131 BEGIN TRANSACTION;
132 DECLARE @ORDER_ID INT = 14;
133 EXEC SHIPPING_DETAILS_INSERT @Order_id = @ORDER_ID;
134
135
136
137
138
```

The bottom pane shows the Results tab with the following data:

OrderID	ProductID	Shipped_Date	Tracking_ID	Delivery_Status	Delivery_Date	Last_Update	
19	13	20	2024-11-26 00:58:32.707	1030	In transit	NULL	2024-11-26 00:58:32.707
20	14	1	2024-11-26 01:04:32.950	1031	In transit	NULL	2024-11-26 01:04:32.950
21	14	5	2024-11-26 01:04:32.950	1032	In transit	NULL	2024-11-26 01:04:32.950
22	14	6	2024-11-26 01:04:32.950	1033	In transit	NULL	2024-11-26 01:04:32.950
23	14	10	2024-11-26 01:04:32.950	1034	In transit	NULL	2024-11-26 01:04:32.950

--After the Delivery agent delivered the Products

-- Stored Procedure to update the delivery status after the product is delivered

```
CREATE PROCEDURE UPDATE_DELIVERY_STATUS
@DELIVERY_STATUS VARCHAR(50), @ORDER_ID INT, @PRODUCT_ID INT
AS
BEGIN
UPDATE Shipped_Products_Details
SET Delivery_Status = @DELIVERY_STATUS, Delivery_Date = GETDATE(), Last_Update =
GETDATE()
WHERE OrderID = @ORDER_ID AND ProductID = @PRODUCT_ID;
END;
```

--Updation of the delivery status transaction begins

```
BEGIN TRANSACTION;
DECLARE @DELIVERY_STATS VARCHAR(50)= 'Delivered';
DECLARE @ORDER_ID INT = 14;
DECLARE @PROD_ID INT = 1;
EXEC UPDATE_DELIVERY_STATUS @DELIVERY_STATUS = @DELIVERY_STATS ,@Order_id =
@ORDER_ID, @PRODUCT_ID = @PROD_ID;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The left pane shows the Object Explorer with the database structure of 'AJAY\SQLEXPRESS (SQL Server 16.0.)'. The central pane shows the execution of a stored procedure named 'UPDATE_DELIVERY_STATUS' in the 'project_ca3' database. The procedure is designed to update the delivery status of a product in the 'Shipped_Products_Details' table. The right pane shows the results of the execution, which includes a table with columns: OrderID, ProductID, Shipped_Date, Tracking_ID, Delivery_Status, Delivery_Date, and Last_Update. The results show that the delivery status for OrderID 14 and ProductID 1 has been updated to 'Delivered'.

OrderID	ProductID	Shipped_Date	Tracking_ID	Delivery_Status	Delivery_Date	Last_Update
19	13	20	2024-11-26 00:58:32.707	1030	In transit	2024-11-26 00:58:32.707
20	14	1	2024-11-26 01:04:32.950	1031	Delivered	2024-11-26 01:07:56.107
21	14	5	2024-11-26 01:04:32.950	1032	In transit	2024-11-26 01:04:32.950
22	14	6	2024-11-26 01:04:32.950	1033	In transit	2024-11-26 01:04:32.950
23	14	10	2024-11-26 01:04:32.950	1034	In transit	2024-11-26 01:04:32.950

8.3.4 Exchange the products

-- Exchanging the products by customer

--Procedure to place exchange request

```
CREATE PROCEDURE EXCHANGE_PRODUCTS
@OrderID int, @ProductID int, @Exchanged_Reason VARCHAR(255)
AS
BEGIN
DECLARE @Exchanged_Status VARCHAR(30) = 'Pending';
INSERT INTO EXCHANGED_PRODUCTS(OrderID, ProductID, Exchanged_Status,
Exchanged_Reason)
SELECT OrderID, ProductID, @Exchanged_Status, @Exchanged_Reason FROM
ORDERED_PRODUCT_DETAILS
WHERE OrderID = @OrderID and ProductID = @ProductID;
END;
```

--Exchange request placement transaction begins

```
BEGIN TRANSACTION;
DECLARE @ORDER_ID INT = 14;
DECLARE @PRODUCT_ID INT = 1;
DECLARE @EXCHANG_REASON VARCHAR(255)= 'Defect';
EXEC EXCHANGE_PRODUCTS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID,
@Exchanged_Reason = @EXCHANG_REASON;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The main window shows a SQL query being executed in the 'Query Editor'. The query is a stored procedure call for 'EXCHANGE_PRODUCTS' with parameters: @OrderID = 14, @ProductID = 1, and @Exchanged_Reason = 'Defect'. Below the query editor, the 'Results' tab is active, showing a table with 8 columns: OrderID, ProductID, Request_Date, Exchanged_Status, Pickup_Date, Exchanged_TrackingID, Exchanged_Reason, and Arrival_Date. The table contains 7 rows of data, with the last row (OrderID 14) showing a 'Pending' status and NULL values for Pickup_Date, Exchanged_TrackingID, and Arrival_Date.

OrderID	ProductID	Request_Date	Exchanged_Status	Pickup_Date	Exchanged_TrackingID	Exchanged_Reason	Arrival_Date
3	5	2024-11-05 01:35:28.290	Picked up	2024-11-05 01:39:08.560	EXTRK001	Defect	2024-11-05 01:40:45.627
4	7	2024-11-09 01:45:46.187	Picked up	2024-11-09 01:47:28.180	EXTRK001	Defect	2024-11-09 01:48:46.217
5	10	2024-11-09 10:01:18.823	Picked up	2024-11-09 10:02:00.680	EXTRK001	Defect	2024-11-09 10:03:35.787
6	13	2024-11-25 22:45:07.597	Picked up	2024-11-25 22:45:27.560	EXTRK0010	Defect	2024-11-25 22:51:24.007
7	14	2024-11-26 01:11:29.923	Pending	NULL	NULL	Defect	NULL

--After the Delivery agent picked up the Products

--Stored Procedure to update the exchange status

```
CREATE PROCEDURE UPDATE_EXCHANGE_PICKUP_STATUS @OrderID INT, @ProductID INT,
@Exchanged_TrackingID VARCHAR(30)
AS
BEGIN
DECLARE @Exchanged_Status_Pickup VARCHAR(30) = 'Picked up';
UPDATE EXCHANGED_PRODUCTS
SET Exchanged_Status = @Exchanged_Status_Pickup, Pickup_Date = GETDATE(),
Exchanged_TrackingID = @Exchanged_TrackingID
WHERE OrderID = @OrderID AND ProductID = @ProductID;
END;
```

--Exchange status after pickup transaction begins

```
BEGIN TRANSACTION;
DECLARE @ORDER_ID INT = 14;
DECLARE @PRODUCT_ID INT = 1;
DECLARE @EXHCNG_TRACKINGID VARCHAR(30) = 'EXTRK0012';
EXEC UPDATE_EXCHANGE_PICKUP_STATUS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID,
@Exchanged_TrackingID = @EXHCNG_TRACKINGID;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The title bar indicates the connection to 'AJAY\SQLEXPRESS.project_ca3 (AJAY\ajay (53))'. The main window shows a SQL query in the 'Query Editor' pane, which is a stored procedure named 'UPDATE_EXCHANGE_PICKUP_STATUS'. The query is as follows:

```
215 BEGIN
216 DECLARE @Exchanged_Status_Pickup VARCHAR(30) = 'Picked up';
217 UPDATE EXCHANGED_PRODUCTS
218 SET Exchanged_Status = @Exchanged_Status_Pickup, Pickup_Date = GETDATE(), Exchanged_TrackingID = @Exchanged_TrackingID
219 WHERE OrderID = @OrderID AND ProductID = @ProductID;
220 END;
221
222
223
224 BEGIN TRANSACTION;
225 DECLARE @ORDER_ID INT = 14;
226 DECLARE @PRODUCT_ID INT = 1;
227 DECLARE @EXHCNG_TRACKINGID VARCHAR(30) = 'EXTRK0011';
228 EXEC UPDATE_EXCHANGE_PICKUP_STATUS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID, @Exchanged_TrackingID = @EXHCNG_TRACKINGID;
229 COMMIT;
230
231 -- Select * from EXCHANGED_PRODUCTS
232
233
234
```

The 'Object Explorer' on the left shows the database structure, including 'Databases' and 'Security'. The 'Results' pane at the bottom displays the output of the query, which is a table with 9 columns: OrderID, ProductID, Request_Date, Exchanged_Status, Pickup_Date, Exchanged_TrackingID, Exchanged_Reason, and Amival_Date. The table contains 7 rows of data.

OrderID	ProductID	Request_Date	Exchanged_Status	Pickup_Date	Exchanged_TrackingID	Exchanged_Reason	Amival_Date
3	5	2024-11-05 01:35:28.290	Picked up	2024-11-05 01:39:08.560	EXTRK001	Defect	2024-11-05 01:40:45.627
4	7	2024-11-09 01:45:46.187	Picked up	2024-11-09 01:47:28.180	EXTRK001	Defect	2024-11-09 01:48:46.217
5	10	2024-11-09 10:01:18.823	Picked up	2024-11-09 10:02:00.680	EXTRK001	Defect	2024-11-09 10:03:35.787
6	13	2024-11-25 22:45:07.597	Picked up	2024-11-25 22:45:27.560	EXTRK0010	Defect	2024-11-25 22:51:24.007
7	14	2024-11-26 01:11:29.923	Picked up	2024-11-26 01:19:58.320	EXTRK0011	Defect	NULL

--After the seller receive the product

--Stored Procedure to update the date of arrival by the seller

```
CREATE PROCEDURE UPDATE_EXCHANGE_ARRIVAL_DATE @OrderID INT, @ProductID INT
AS
BEGIN
UPDATE Exchanged_Products
SET Arrival_Date = GETDATE()
WHERE OrderID = @OrderID and ProductID = @ProductID;
END;
```

--Updation of the product arrival date transaction begins

```
BEGIN TRANSACTION;
DECLARE @ORDER_ID INT = 14;
DECLARE @PRODUCT_ID INT = 1;
EXEC UPDATE_ARRIVAL_DATE @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The title bar indicates the connection is to 'AJAY\SQLEXPRESS,project_ca3 (AJAY\ajay (53))'. The Object Explorer on the left shows the database structure, including 'Databases' (System Databases, Database Snapshots, advance, CA3, dbms, mavenmovies, new_sca, practice, present, project_ca3, SCA, sysdb) and 'Server Objects' (Security, Replication, Management, XEvent Profiler). The query window, titled 'product_buy_code.sql - not connected', contains the following SQL code:

```
245 BEGIN
246 UPDATE Exchanged_Products
247 SET Arrival_Date = GETDATE()
248 WHERE OrderID = @OrderID and ProductID = @ProductID;
249 END;
250
251
252
253
254
255 BEGIN TRANSACTION;
256 DECLARE @ORDER_ID INT = 14;
257 DECLARE @PRODUCT_ID INT = 1;
258 EXEC UPDATE_ARRIVAL_DATE @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID;
259 COMMIT;
260
261 --- Select * from Exchanged_Products;
262
263
264
```

The Results pane at the bottom shows a grid with 9 columns: OrderID, ProductID, Request_Date, Exchanged_Status, Pickup_Date, Exchanged_TrackingID, Exchanged_Reason, and Arrival_Date. The data is as follows:

OrderID	ProductID	Request_Date	Exchanged_Status	Pickup_Date	Exchanged_TrackingID	Exchanged_Reason	Arrival_Date	
3	5	10	2024-11-05 01:35:28.290	Picked up	2024-11-05 01:39:08.560	EXTRK001	Defect	2024-11-05 01:40:45.627
4	7	10	2024-11-09 01:45:46.187	Picked up	2024-11-09 01:47:28.180	EXTRK001	Defect	2024-11-09 01:48:46.217
5	10	8	2024-11-09 10:01:18.823	Picked up	2024-11-09 10:02:00.680	EXTRK001	Defect	2024-11-09 10:03:35.787
6	13	1	2024-11-25 22:45:07.597	Picked up	2024-11-25 22:45:27.560	EXTRK0010	Defect	2024-11-25 22:51:24.007
7	14	1	2024-11-26 01:11:29.923	Picked up	2024-11-26 01:19:58.320	EXTRK0011	Defect	2024-11-26 01:24:36.120

8.3.5 Return the products

-- Returning the products by customer

--Stored Procedure to place return request

```
CREATE PROCEDURE RETURN_PRODUCTS @OrderID int, @ProductID int, @Return_Reason  
VARCHAR(255)
```

```
AS
```

```
BEGIN
```

```
DECLARE @Returned_Status VARCHAR(30) = 'Pending';
```

```
DECLARE @Refund_Status VARCHAR(30) = 'Pending';
```

```
INSERT INTO Returned_Products(OrderID, ProductID, Returned_Status, Refund_Status,  
Returned_Reason)
```

```
SELECT OrderID, ProductID, @Returned_Status, @Refund_Status, @Return_Reason FROM  
ORDERED_PRODUCT_DETAILS
```

```
WHERE OrderID = @OrderID and ProductID = @ProductID;
```

```
END;
```

--Return request placement transaction starts

```
BEGIN TRANSACTION;
```

```
DECLARE @ORDER_ID INT = 14;
```

```
DECLARE @PRODUCT_ID INT = 5;
```

```
DECLARE @RETURNREASON VARCHAR(255) = 'Defect';
```

```
EXEC RETURN_PRODUCTS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID,
```

```
@Return_Reason = @RETURNREASON;
```

```
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The 'Object Explorer' on the left shows the database structure for 'project_ca3'. The 'Query Editor' in the center shows the execution of the 'RETURN_PRODUCTS' stored procedure with the following SQL code:

```
278 DECLARE @Refund_Status VARCHAR(30) = 'Pending';  
279 INSERT INTO Returned_Products(OrderID, ProductID, Returned_Status, Refund_Status, Returned_Reason)  
280 SELECT OrderID, ProductID, @Returned_Status, @Refund_Status, @Return_Reason FROM ORDERED_PRODUCT_DETAILS  
281 WHERE OrderID = @OrderID and ProductID = @ProductID;  
282 END;  
283 -----  
284 -----  
285 -----  
286 -----  
287 BEGIN TRANSACTION;  
288 DECLARE @ORDER_ID INT = 14;  
289 DECLARE @PRODUCT_ID INT = 5;  
290 DECLARE @RETURNREASON VARCHAR(255) = 'Defect';  
291 EXEC RETURN_PRODUCTS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID, @Return_Reason = @RETURNREASON;  
292 COMMIT;  
293 -----  
294 -----  
295 -----  
296 -----  
297 -----
```

The 'Results' pane at the bottom shows the output of the query, which is a table with 9 columns: OrderID, ProductID, Request_Date, Returned_Status, Pickup_Date, Refund_Status, Returned_TrackingID, Returned_Reason, and Arrival_Date. The table contains 8 rows of data, with the last row (OrderID 14, ProductID 5) showing a 'Pending' status and a 'NULL' arrival date.

OrderID	ProductID	Request_Date	Returned_Status	Pickup_Date	Refund_Status	Returned_TrackingID	Returned_Reason	Arrival_Date
4	5	2024-11-05 01:47:37.767	Picked up	2024-11-05 01:51:04.913	Paid	RTRK001	Defect	2024-11-05 01:56:11.220
5	7	2024-11-09 01:51:11.967	Picked up	2024-11-09 01:52:48.013	Paid	RTRK005	Defect	2024-11-09 01:53:55.240
6	10	2024-11-09 10:05:50.020	Picked up	2024-11-09 10:07:27.783	Paid	RTRK008	Defect	2024-11-09 10:07:54.390
7	13	2024-11-25 22:54:23.627	Picked up	2024-11-25 23:06:55.053	Paid	RTRK012	Defect	2024-11-25 23:08:51.130
8	14	2024-11-26 01:30:16.623	Pending	NULL	Pending	NULL	Defect	NULL

--After the Delivery agent picked up the Products

--Stored Procedure to update returned product pickup status

```
CREATE PROCEDURE UPDATE_RETURNED_PICKUP_STATUS @OrderID INT, @ProductID INT,
@Returned_TrackingID VARCHAR(30)
AS
BEGIN
DECLARE @Returned_Status VARCHAR(30) = 'Picked_up';
DECLARE @Refund_Status VARCHAR(30) = 'Paid';
UPDATE Returned_Products
SET Returned_Status = @Returned_Status, Pickup_Date = GETDATE(), Refund_Status =
@Refund_Status,
Returned_TrackingID = @Returned_TrackingID
WHERE OrderID = @OrderID AND ProductID = @ProductID;
END;
```

--Pickup status transaction starts

```
BEGIN TRANSACTION;
DECLARE @ORDER_ID INT = 14;
DECLARE @PRODUCT_ID INT = 5;
DECLARE @RETURN_TRACKINGID VARCHAR(30) = 'RTRK013';
EXEC UPDATE_RETURNED_PICKUP_STATUS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID,
@Returned_TrackingID = @RETURN_TRACKINGID;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The 'Object Explorer' on the left shows the database structure for 'project_ca3'. The main query window, titled 'major_project_code....a3 (AJAY\ajay (53))', contains the following SQL code:

```
312 SET Returned_Status = @Returned_Status, Pickup_Date = GETDATE(), Refund_Status = @Refund_Status,
313 Returned_TrackingID = @Returned_TrackingID
314 WHERE OrderID = @OrderID AND ProductID = @ProductID;
315 END;
316 -----
317 -----
318 -----
319 -----
320 BEGIN TRANSACTION;
321 DECLARE @ORDER_ID INT = 14;
322 DECLARE @PRODUCT_ID INT = 5;
323 DECLARE @RETURN_TRACKINGID VARCHAR(30) = 'RTRK013';
324 EXEC UPDATE_RETURNED_PICKUP_STATUS @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID, @Returned_TrackingID = @RETURN_TRACKINGID;
325 COMMIT;
326 -----
327 -- Select * from Returned_Products;
328 -----
329 -----
330 -----
331 -----
```

Below the query window, the 'Results' tab shows a grid of data with the following columns: OrderID, ProductID, Request_Date, Returned_Status, Pickup_Date, Refund_Status, Returned_TrackingID, Returned_Reason, and Arrival_Date. The data is as follows:

OrderID	ProductID	Request_Date	Returned_Status	Pickup_Date	Refund_Status	Returned_TrackingID	Returned_Reason	Arrival_Date
4	5	2024-11-05 01:47:37.767	Picked up	2024-11-05 01:51:04.913	Paid	RTRK001	Defect	2024-11-05 01:56:11.220
5	7	2024-11-09 01:51:11.967	Picked up	2024-11-09 01:52:48.013	Paid	RTRK005	Defect	2024-11-09 01:53:55.240
6	10	2024-11-09 10:05:50.020	Picked up	2024-11-09 10:07:27.783	Paid	RTRK008	Defect	2024-11-09 10:07:54.390
7	13	2024-11-25 22:54:23.627	Picked up	2024-11-25 23:06:55.053	Paid	RTRK012	Defect	2024-11-25 23:08:51.130
8	14	2024-11-26 01:30:16.623	Picked up	2024-11-26 01:32:00.500	Paid	RTRK013	Defect	NULL

--After the seller got the product

--Stored Procedure to update returned product arrival date by the seller

```
CREATE PROCEDURE UPDATE_RETURNED_ARRIVAL_DATE @OrderID INT, @ProductID INT
AS
BEGIN
UPDATE Returned_Products
SET Arrival_Date = GETDATE()
WHERE OrderID = @OrderID AND ProductID = @ProductID;
END;
```

--Transaction to update receive date of the returned products by the seller

```
BEGIN TRANSACTION;
DECLARE @ORDER_ID INT = 14;
DECLARE @PRODUCT_ID INT = 5;
EXEC UPDATE_RETURNED_ARRIVAL_DATE @OrderID = @ORDER_ID, @ProductID = @PRODUCT_ID;
COMMIT;
```

The screenshot displays the Microsoft SQL Server Management Studio interface. The left pane shows the 'Object Explorer' with a tree view of the 'AJAY\SQLEXPRESS (SQL Server 16.0)' instance, including databases like 'CA3', 'dbms', 'mavenmovies', 'new_sca', 'practice', 'present', 'project_ca3', 'SCA', and 'sysdb'. The central pane shows a SQL query being executed in the 'major_project_code...a3 (AJAY\ajay (53))' window. The query is a stored procedure that updates the arrival date of returned products and a transaction to execute this procedure. The bottom pane shows the 'Results' tab with a table of returned products.

OrderID	ProductID	Request_Date	Returned_Status	Pickup_Date	Refund_Status	Returned_TrackingID	Returned_Reason	Arrival_Date
4	5	2024-11-05 01:47:37.767	Picked up	2024-11-05 01:51:04.913	Paid	RTRK001	Defect	2024-11-05 01:56:11.220
5	7	2024-11-09 01:51:11.967	Picked up	2024-11-09 01:52:48.013	Paid	RTRK005	Defect	2024-11-09 01:53:55.240
6	10	2024-11-09 10:05:50.020	Picked up	2024-11-09 10:07:27.783	Paid	RTRK008	Defect	2024-11-09 10:07:54.390
7	13	2024-11-25 22:54:23.627	Picked up	2024-11-25 23:06:55.053	Paid	RTRK012	Defect	2024-11-25 23:08:51.130
8	14	2024-11-26 01:30:16.623	Picked up	2024-11-26 01:32:00.500	Paid	RTRK013	Defect	2024-11-26 01:39:33.340

9. Conclusion

The project successfully implements an e-commerce database management system to efficiently handle customers, products, orders, and shipping information. The system ensures scalable and normalized data storage, allowing smooth transactions, and retrieval of key data for further analysis.